一、cookie_session

二、教学目标

1、掌握forms组件的Widget参数使用

2、理解Cookie和Session的概念与区别

3、学会在Django中操作Cookie

4、掌握使用装饰器实现登录验证

5、学会在Django中操作Session

三、forms组件

3.0.1 Widget 相关参数

虽然 Widget 参数不是字段本身的参数，但通常通过字段的 `widget` 参数传递 Widget 实例，并在 Widget 的 `attrs` 中设置 HTML 属性。例如：

```
name = forms.CharField(
    widget=forms.TextInput(attrs={
        'class': 'form-control',
        'placeholder': 'Enter your name'
    })
)
```

应用

```
class MyForm(forms.Form):
    ......

    password = forms.CharField(min_length = 3, max_length = 8,label='密码',
                                    error_messages={
                                        'required': '密码不能为空',
                                        'min_length':'密码最小3位',
                                        'max_length': '密码最大8位'
                                    },
                                    widget=forms.PasswordInput(attrs=
{'class':'form-control'})
                                )
```

radio 单选框

```
# initial 默认值
    # choices 枚举
    # widget 样式
    gender = forms.ChoiceField(label='性别',
                               choices=((1, '男'), (2, '女'), (3, '保密'))
                               , initial=3
                               , widget=forms.RadioSelect()

    )
```

select 下拉单选框

```
hobby = forms.ChoiceField(label='爱好', choices=((1, '唱'), (2, '跳'), (3,
'rap')), widget=forms.Select())
```

select 下拉多选框

```
hobby = forms.ChoiceField(label='爱好', choices=((1, '唱'), (2, '跳'), (3,
'rap')), widget=forms.SelectMultiple())
```

checkbox单选框

```
remember = forms.ChoiceField(label='是否记住密
码',initial='checked',widget=forms.CheckboxInput())
```

checkbox多选框

```
# checkbox多选框
hobby = forms.ChoiceField(label='爱好', choices=((1, '唱'), (2, '跳'), (3,
'rap')), widget=forms.CheckboxSelectMultiple())
```

四、cookie 和 session

- 会话-从打开浏览器访问一个网站，到关闭浏览器结束此次访问，称之为一次会话
- HTTP协议是无状态的，导致会话状态难以保持
- 试想一下，如果不保持会话状态，在电商网站购物的场景体验?

cookies和Session就是为了保持会话状态而诞生的两个存储技术

4.1 cookie

- **cookies是保存在客户端浏览器上的存储空间**
    - Chrome 浏览器 可能通过开发者工具的 Application >>storage >> cookies 查看和操作浏览器端所有的 Cookies 值

    - 火狐浏览器 可能通过开发者工具的 存储 ->Cookie
- **cookies 特点**
    - cookies 在浏览器上是以键-值对的形式进行存储的，键和值都是以ASCII字符串的形存储(不能是中文字符串)
    - cookies中的数据是按域存储隔离的，不同的域之间自动区别，不会访问
    - cookies 的内部的数据会在每次访问此网址时都会携带到服务器端，如果cookies过大会降低响应速度

小总结:

cookie：就是一段小信息,服务器将登录信息以键值对的形式发送浏览器进行保存,之后每次访问浏览器就可以自动校验,这种方式存在一定的安全隐患

- **在Django 设置浏览器的COOKIE 必须通过 HttpResponse 对象来完成**
    - **添加、修改COOKIE**
        - HttpResponse.set_cookie(key, value='', max_age=None,expires=None)
        - key:cookie的名字
        - value:cookie的值
        - max_age:cookie存活时润，秒为单位
        - expires:具体过期时间
        - 当不指定max_age和expres 时,关闭浏览器时此数据失效
    - **删除COOKIE**
        - HttpResponse.delete_cookie(key)
        - 删除指定的key 的Cookie。如果key 不存在则什么也不发生。
    - **获取cookie**
        - 通过 request.COOKIES 绑定的字典(dict) 获取客户端的COOKIES数据

```
value = request.COOKIES.get('cookies名','默认值')
```

4.1.1 添加cookie

urls.py

```
path('login/',views.login),
path('home/',views.home)
```

app01 -> views.py

```
def login(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        # 简单的硬编码验证先 有需求的可以用数据库
        if username == 'admin' and password == '123456':
            # return redirect('home')
            response = redirect('/home/')
            # 设置Cookie，名为'username'，值为用户名，有效期为1天（单位秒）
            response.set_cookie('username', username, max_age=86400)
            return response
        else:
            error_msg = '用户名或密码错误！'
```

```
            return render(request, 'login.html', {'error_msg': error_msg})
    return render(request,'login.html')


def home(request):
    # 从Cookie中获取用户名
    username = request.COOKIES.get('username')
    if username:
        return render(request, 'home.html', locals())
    else:
        return redirect('/login/')
```

login.html

```
<body>
<form action="" method="post">
    <p>用户名: <input type="text" name="username"></p>
    <p>密码: <input type="text" name="password"></p>
    <input type="submit">
</form>
</body>
```

home.html

```
<body>
<h2>欢迎{{ username }}</h2>
<p>您已成功登录。</p>
</body>
```

如果我当前项目有很多的页面是不是意味着我所有的视图函数中都要用到这个cookie的验证?

这样做显然不适合 我们可以把它写成-**登录装饰器**

4.1.2 登录装饰器

urls.py

```
 path('page/', views.page)
```

app01 -> views.py

```
# 登录认证装饰器
def login_auth(func):

    def inner(request,*args,**kwargs):
        username = request.COOKIES.get('username')
        if username == 'admin':
            return func(request,*args,**kwargs)
        else:
            return redirect('/login/')

    return inner

@login_auth
def home(request):
```

```
    # 从Cookie中获取用户名 / 从Cookie获取用户名并传递给模板
    username = request.COOKIES.get('username')
    # if username == 'admin':
    #     return render(request, 'home.html', locals())
    # else:
    #     return redirect('/login/')
    return render(request, 'home.html', locals())


@login_auth
def page(request):
    return render(request, 'page.html')
```

page.html

```
<p>我是page页面</p>
```

问题出现:

比如你现在输入page页面 然后登录认证器拦截到没cookie的时候重新登录成功后到达home页面

正确做法应该是我在哪个页面跳转登录页面的登录成功后应该跳回哪里去

```
def login(request):
    ....
            response = redirect('/home/')
           ....
        else:
       .....
    return render(request,'login.html')
```

出现问题的代码就是在登录成功后 重定向固定到home页面了

改进:

```
def login_auth(func):

    def inner(request,*args,**kwargs):
        username = request.COOKIES.get('username')
        url = request.get_full_path() #获取用户访问的url的路径
        if username == 'admin':
            return func(request,*args,**kwargs)
        else:
            return redirect('/login/?next=%s' %url)

    return inner
```

在登录认证装饰器中先获取到url 把参数传递给login

```
def login(request):
    if request.method == 'POST':
        .....
        if username == 'admin' and password == '123456':
            # return redirect('home')
             url = request.GET.get('next')
            if not url:  # 如果url为None，设置默认跳转到home页面
                url = '/home/'
            response = redirect(url)
           .....
        else:
           ........
    return render(request,'login.html')
```

总结:

登录装饰器-> 判断用户是否登录,只有登录之后才能进入某些页面,如果没有登录就跳转到登录页面用户在没有登录的时候访问某个页面,跳转到登录页面,登录之后应该重定向到之前的访问的页面

4.1.3 删除cookie

home.html

```
<body>
<h2>欢迎{{ username }}</h2>
<p>您已成功登录。</p>
<a href="{% url 'logout' %}">退出登录</a>
</body>
```

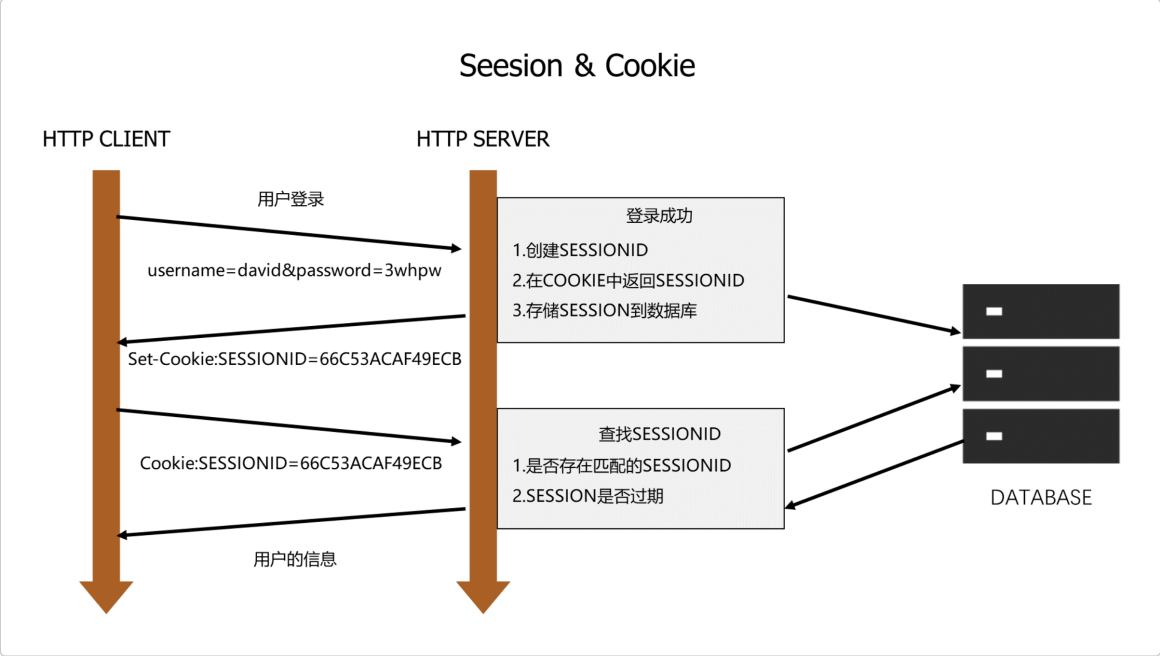
urls.py

```
path('logout/', views.logout, name='logout'),
```

app01 -> views.py

```
@login_auth
def logout(request):
    #注销 重定向到登录页面
    response = redirect('/login/')
    response.delete_cookie('username')
    return response
```

4.2 session



- session是保存在服务器端的存储空间（通常保存在服务器内存、文件、**数据库**或缓存中）。
- session的工作机制：
  1. 当客户端第一次访问服务器时，服务器会为当前会话创建一个session对象，并生成一个唯一的session id（通常是一个字符串）。
  2. 服务器在HTTP响应中通过Set-Cookie头将session id发送给客户端，客户端会将其保存在cookie中（默认情况下，该cookie的名称为sessionid，但也可配置）。
  3. 客户端在后续的请求中会自动携带这个包含session id的cookie，服务器通过这个session id来识别用户并从存储中取出对应的session数据。
- session特点：
  - session数据存储在服务器端，相对安全，适合存储敏感信息（如用户登录状态、用户信息等）。
  - session依赖于cookie（或者URL重写）来传递session id。如果客户端禁用了cookie，则需要通过URL重写的方式将session id嵌入URL中（但这种方式不够安全且不方便）。
  - session会占用服务器资源，如果用户量很大，服务器需要存储大量的session数据，因此需要考虑存储方式（如使用数据库或缓存）和清理机制（如设置过期时间）。

小总结:

session:数据保留在服务器上,服务器产生一个随机字符串交给浏览器以键值对的形式保存.之后访问网站,浏览器会自动带着随机字符串到服务端进行校验.

4.2.1 添加session

使用db.sqlite3自带的数据库保存session

```
python manage.py migrate
```

不写模型类,也可以直接进行数据库迁移,会自动生成 10 多张表,有一张叫做 django_session的表是用来保存 session 数据的

urls.py

```
path('set_session/', views.set_session),
```

app01 -> views.py

```
def set_session(request):
    request.session['hoddy'] = 'sing'
    return HttpResponse('设置成功')
```

设置 session

- django 内部会自动生成一个随机字符串
- 将这个随机字符串和对应的数据(加密)保存在 django_session 表中
- 将随机字符串交给浏览器保存

4.2.2 获取session

urls.py

```
path('get_session/', views.get_session),
```

app01 -> views.py

```
def get_session(request):
    res = request.session.get('hobby')
    return HttpResponse('你的爱好是%s'%res)
```

获取 session

- 自动从请求的浏览器中获取 sessionid 对应的值(随机字符串)
- 拿着随机字符串到 django_session 表中查找对应的数据
- 如果比对上了就取出对应的数据封装到request.session 中,如果对比不上就会返回None

那如果设置多个session呢?

app01 -> views.py

```
def set_session(request):
    request.session['hobby'] = 'sing'
    request.session['hobby1'] = 'dance'
    request.session['hobby2'] = 'rap'
    return HttpResponse('设置成功')

def get_session(request):
    res1 = request.session.get('hobby')
    res2 = request.session.get('hobby1')
    res3 = request.session.get('hobby2')
    return HttpResponse('你的爱好是%s,%s,%s'%(res1,res2,res3))
```

在同一台计算机的同一个浏览器,不管怎么设置 session,在 django_session 表中都只会有一条数据,除非保存的session过期了,才会产生新的数据,主要是为了节省服务器的数据库资源

4.2.3 失效session

session 设置过期时间

request.session.set_expiry()

参数:

- 整数：多少秒之后过期
- 日期：指定过期日期
- 0：关闭浏览器就过期
- 不写：取决于 django 默认的过期时间(14 天)

```
def set_session(request):
    request.session['hobby'] = 'sing'
    request.session['hobby1'] = 'dance'
    request.session['hobby2'] = 'rap'
    #10秒
    request.session.set_expiry(10)

    return HttpResponse('设置成功')
```

4.2.4 删除session

urls.py

```
path('del_session/', views.del_session),
```

app01 -> views.py

```
def del_session(request):
    #只删除数据库的session 不删除浏览器的 不推荐使用
    #request.session.delete()
    #将数据库的浏览器的数据都删除
    request.session.flush()
    return HttpResponse('删除了session')
```

五、作业

5.1 完成本天的案例